Архитектура BIOS Aleste

Дизайн для реального железа

h2w

26 декабря 2025 г.

Содержание

1. Архитектура BIOS Aleste: Дизайн для реального железа **1**
- 1.1. Философия: Скорость через предсказуемость . . . 2
- 1.2. Структура драйвера: Один файл — одно устройство . . . 2
- 1.3. Полиморфные драйверы в BIOS Aleste: Два паттерна использования VTable . . . 3
- 1.4. Table API: Минимум обязательного, максимум возможного . . . 4
- 1.5. Типизация команд: Избегаем хаоса . . . 5
- 1.6. Информация о драйвере: Система должна знать, с кем имеет дело . . . 6
- 1.7. Горячее обнаружение: Для тех, кто может исчезнуть . . . 6
- 1.8. Прерывания: Быстро или безопасно — выбор разработчика . . . 7
- 1.9. Зависимости: Явное лучше неявного . . . 8
- 1.10. Контекст памяти: Lazy switching . . . 9
- 1.11. Соглашения о регистрах: Естественность для Z80 . . . 9
- 1.12. Пример из реальной жизни: Аудиодрайвер для YM2149 . . . 10
- 1.13. Структура проекта: Как это всё собирается . . . 13

1. Архитектура BIOS Aleste: Дизайн для реального железа

Введение: Зачем нам нужен новый стандарт?

Когда пишешь код для Z80 на реальном железе вроде Aleste, каждая инструкция на счету. Существующие стандарты часто приходят с больших платформ и не учитывают специфику 8-битных систем. Мы создаем не абстракцию ради абстракции, а практичную систему, где:

- Прямой вызов драйвера стоит ровно 17 тактов
- Переключение банков памяти происходит только когда действительно нужно
- Обработчик прерываний аудио не тратит время на сохранение регистров, которые не использует

Этот документ — не просто спецификация. Это философия разработки для ограниченных ресурсов.

1.1. Философия: Скорость через предсказуемость

1. Фиксированные слоты вместо поиска

   Представьте, что вам нужно вывести пиксель на экран. В сложных системах вы бы:

   a) Искали драйвер видео в таблице

   b) Запрашивали функцию вывода

   c) Вызывали её через указатель

   В нашей системе вы просто пишете:

   ```
   call 0xFD03  ; video_draw_pixel
   ```

   Почему это важно: 17 тактов против 50+. При частоте кадров 50 Гц у вас есть всего 20 мс на кадр. Каждая экономия тактов — это возможность добавить больше спрайтов, больше эффектов.

2. Реальные цифры:

   - Стандартный вызов через фиксированный слот: **17 тактов**
   - Вызов через таблицу указателей: ~50 тактов
   - Динамический поиск в runtime: 100+ тактов

1.2. Структура драйвера: Один файл — одно устройство

Мы отказались от разделения на `_api.asm` и `_impl.asm`. Почему?
Проблема: Когда драйвер раскидан по нескольким файлам:

1. Ты открываешь `video_api.asm`, видишь прототипы
2. Переходишь в `video_impl.asm`, ищешь реализацию
3. Забываешь, где лежат данные драйвера

Наше решение: Открой `drivers/video/crtc.asm` — и увидишь всё:

```
; В начале файла — таблица переходов
SECTION DRV_VTABLE
    ; луче устанавливать allign.
    allign 32
crtc_vtable:
    jp crtc_detect
    jp crtc_install
    ...

; Прямо здесь же — данные драйвера
SECTION DRV_DATA
crtc_mode: db 0
crtc_buffer: ds 256

; И сразу код
SECTION DRV_CODE
crtc_detect:
```

```
; проверка наличия CRTC
...
```

Результат: Локальность. Меньше прыжков по файлам, меньше ошибок.

1.3. Полиморфные драйверы в BIOS Aleste: Два паттерна использования VTable

В системе существуют два типа драйверов с разными схемами использования VTable:

1. Синглтон-драйверы (статичные, неделимые)

   Это драйверы для уникального железа, которое не имеет альтернатив:

   - **MMU** - только один тип маппера в системе
   - **CRTC** - конкретный видеоконтроллер
   - **PSG** - конкретный звуковой чип

   Структура:

   ```
   ; Единая таблица переходов
   PUBLIC _mmu_vtable, _mmu_detect, _mmu_install

   _mmu_vtable:
   _mmu_detect:    jp mmu_detect_impl
   _mmu_install:   jp mmu_install_impl
   ```

   Использование:

   - В заголовочном файле: прямые вызовы по именам меток
   - В коде: `call _mmu_install`
   - VTable используется только системой при загрузке

2. Полиморфные драйверы (многорежимные, заменяемые)

   Это драйверы, имеющие несколько реализаций для одного интерфейса:

   - **Видео** - разные видеорежимы (текст, графика)
   - **Консоль** - разные кодировки или шрифты
   - **Файловая система** - FAT12, FAT16, TR-DOS

   a) Структура:

   ```
   ; Полиморфная таблица верхнего уровня
   PUBLIC _video_vtable, _video_draw_char, _video_set_mode

   _video_vtable:
   _video_draw_char:    jp 0          ; Заполняется при активации
   _video_set_mode:     jp 0          ; Заполняется при активации

   ; Конкретная реализация например(, текстовый режим)
   PUBLIC _text_vtable
   _text_vtable:
       jp text_draw_char_impl
       jp text_set_mode_impl
   ```

b) Процесс активации:

Конкретная реализация может иметь или не иметь метки, так как чаще всего будет использованп полиморфная таблица. Для этого, когда драйверу устанавливается режим он копирует конкретную таблицу в полиморфную версию.

```
; Система копирует таблицу конкретного драйвера в слот
copy_driver_table:
    ; HL = source (_text_vtable)
    ; DE = destination (_video_vtable)
    ld bc, 15        ; 5 методов × 3 байта
    ldir
```

Использование:

c) Ключевые отличия:

- В заголовочном файле: всё те же прямые вызовы `call _video_draw_char`
- Runtime: вызов автоматически идет в текущую активную реализацию
- Переключение: копирование VTable без изменения вызывающего кода

Аспект	Синглтон-драйвер	Полиморфный драйвер
Количество реализаций	1	Много
VTable в памяти	Фиксированная, никогда не меняется	Меняется при переключении режима
Вызов из кода	Прямой переход по фиксированному адресу	Прямой переход по адресу, который меняет реализацию
Заголовочный файл	Содержит фиксированные адреса методов	Содержит адреса полиморфного слота
Примеры	MMU, таймер, PSG	Видео, консоль, FS

d) Философия:

i. **Единый интерфейс вызова** - код приложения всегда вызывает `_video_draw_char`, независимо от режима

ii. **Нулевые накладные расходы** - вызов всегда 17 тактов, даже при полиморфизме

iii. **Простота переключения** - сменить драйвер = скопировать 15 байт

iv. **Прозрачность** - отладчик видит прямой jump, а не косвенный вызов через регистр

Именно поэтому в документации говорится <<прямой вызов драйвера стоит ровно 17 тактов>> - потому что даже полиморфный вызов это прямой `jp` на конкретный адрес, просто этот адрес может меняться в runtime.

Полиморфизм достигается не через таблицы указателей в памяти (что давало бы +50 тактов), а через физическое копирование инструкций перехода. Это гениально просто для Z80.

1.4. Table API: Минимум обязательного, максимум возможного

1. Обязательные методы (первые 15 байт)

Каждый драйвер обязан реализовать 5 методов:

Метод	Что делает	Когда вызывается
`detect`	Проверяет, присутствует ли устройство	При загрузке и (опционально) периодически
`install`	Инициализирует устройство	После успешного detect
`get_info`	Возвращает информацию о драйвере	По запросу системы или приложения
`command`	Универсальная точка расширения	Когда нужно специфичное действие
`uninstall`	Освобождает ресурсы	При выгрузке драйвера

2. Магия команды `command`

   Вместо того чтобы расширять таблицу прыжков для каждой новой функции:

```
; ПЛОХО: Таблица раздувается
jp video_set_mode
jp video_set_palette
jp video_set_sprite
jp video_scroll
... и ещё 20 функций

; ХОРОШО: Один метод на всё
jp video_command
```

   Как это работает:

```
; Установить видеорежим
ld a, CMD_VID_SET_MODE
ld hl, MODE_256x192
call 0xFD00  ; video_command

; Прокрутить экран
ld a, CMD_VID_SCROLL
ld bc, 10  ; на 10 пикселей
call 0xFD00
```

   Преимущество: Таблица прыжков всегда занимает 15 байт + специфичные методы. Не нужно резервировать слоты под функции, которые драйвер не поддерживает.

1.5. Типизация команд: Избегаем хаоса

1. Диапазоны команд

   Чтобы команды разных драйверов не конфликтовали:

```
0x00-0x0F: Системные все( драйверы)
  0x00 = RESET
  0x01 = STATUS
  0x02 = SUSPEND
  0x03 = RESUME

0x10-0x1F: Видео
```

```
  0x10 = SET_MODE
  0x11 = SET_PALETTE
  0x12 = SCROLL

0x20-0x2F: Аудио
  0x20 = PLAY
  0x21 = STOP
  0x22 = SET_VOLUME

0x30-0x3F: Хранилище
  0x30 = READ_SECTOR
  0x31 = WRITE_SECTOR
```

Пример использования:

```
; Играть музыку
ld a, CMD_AUD_PLAY  ; 0x20
ld hl, song_data
call 0xFD00  ; audio_command

; Остановить
ld a, CMD_AUD_STOP  ; 0x21
call 0xFD00
```

1.6. Информация о драйвере: Система должна знать, с кем имеет дело

1. Структура DRIVER_INFO

```
; Что возвращает get_info
driver_info:
  dw .name        ; "YM2149 Audio Driver"
  db 2, 1         ; Версия 2.1
  dw API_HAS_IRQ | API_STATIC_DEVICE  ; Флаги
  db IRQ_VECTOR   ; Номер вектора прерывания если( нужно)
  db 0            ; Резерв
.name:
  db "YM2149 v2.1", 0
```

2. Флаги возможностей:

```
API_HAoS_IRQ        equ 1 << 0  ; Использует прерывания
API_STATIC_DEVICE  equ 1 << 1  ; Не исчезнет встроенное( устройство)
API_USES_DMA       equ 1 << 2  ; Требует DMA
API_BANKED         equ 1 << 3  ; Работает с банковой памятью
API_POWER_SAVE     equ 1 << 4  ; Поддерживает энергосбережение
```

Зачем это нужно: Система видит флаг `API_STATIC_DEVICE` и понимает — этот видеоконтроллер встроенный, не нужно опрашивать его каждые 100 мс на предмет <<не исчез ли он>>.

1.7. Горячее обнаружение: Для тех, кто может исчезнуть

1. Два типа устройств

a) **Статические** (встроенные): Видеоконтроллер, системный таймер
   - `detect` вызывается один раз при загрузке
   - Флаг `API_STATIC_DEVICE` установлен

b) **Динамические** (сменные): Картриджи, внешние устройства
   - `detect` может вызываться периодически
   - Система отслеживает их наличие

2. Протокол обнаружения исчезновения

```
; Псевдокод ядра
check_hotplug:
    ; Для каждого динамического устройства
    call driver_detect
    jr z, .device_present

    ; Устройство исчезло!
    call driver_uninstall
    mark_slot_free

.device_present:
    ; Всё на месте
    ret
```

Важный момент: `uninstall` должен быть идемпотентным. Если вызвать его дважды — ничего страшного не случится.

1.8. Прерывания: Быстро или безопасно — выбор разработчика

1. Проблема стандартных обработчиков

   Типичный обработчик сохраняет ВСЕ регистры:

```
irq_handler:
    push af
    push bc
    push de
    ...
    ; 100+ тактов только на push/pop!
    pop hl
    pop de
    pop bc
    pop af
    ret
```

   Для аудиодрайвера, который должен обновлять буфер каждые 20 мс, это неприемлемо.

2. Наше решение: Гибкость

   Драйвер регистрирует обработчик как есть. Если он написан на ассемблере и знает, какие регистры использует:

```
; Быстрый обработчик для YM2149
ym_irq_handler:
    push af
    push hl

    ; Обновляем только регистры YM
    ld hl, ym_buffer
    ld a, (hl)
    out (YM_PORT), a

    pop hl
    pop af
    ei
    ret
```

Но предупреждение: Если обработчик написан на C или использует библиотечные функции — он ДОЛЖЕН сохранять все регистры. Система доверяет разработчику.

3. 7.3 Регистрация обработчика

```
; В install драйвера:
ld a, CMD_SYS_IRQ_ATTACH
ld hl, ym_irq_handler
ld bc, IRQ_VECTOR
call SYS_CALL  ; зарегистрировать
```

1.9. Зависимости: Явное лучше неявного

1. Декларация зависимостей

   В начале файла драйвера:

```
; drivers/console/text.asm
DEPENDS_ON:
    db "VIDEO", 0    ; Нужно для вывода
    db "KEYBOARD", 0 ; Нужно для ввода
    db 0             ; Конец списка
```

2. Что происходит при загрузке

   a) Система читает `DEPENDS_ON` каждого драйвера

   b) Проверяет, все ли зависимости доступны

   c) Если нет — выводит понятную ошибку:

```
CONSOLE: Missing dependency: KEYBOARD
```

Преимущество: Не нужно в runtime проверять <<а подключена ли клавиатура?>>. Система знает это заранее.

1.10. Контекст памяти: Lazy switching

1. Проблема банковой памяти

   На Z80 с MMU переключение банков — дорого:

```
; Сохранить текущую конфигурацию
ld a, (current_bank)
push af

; Переключить на банк драйвера
ld a, DRIVER_BANK
out (MMU_PORT), a

; Выполнить операцию
call driver_function

; Восстановить
pop af
out (MMU_PORT), a
```

   ~50 тактов на переключение туда-обратно!

2. Наше решение: Кэширование

   Каждый драйвер хранит свой <<отпечаток пальца>>:

```
; Memory Map драйвера
crtc_mem_map:
    db 0xA3        ; Хэш конфигурации
    db 5, 6, 7, 8  ; Банки для окон
```

   Перед вызовом драйвера:

```
require_memory:
    ; Сравнить хэш текущей конфигурации с хэшем драйвера
    ld a, (current_hash)
    cp (hl)  ; HL указывает на crtc_mem_map
    ret z    ; Уже правильная конфигурация!

    ; Переключить банки
    inc hl
    ld bc, MMU_PORT
    outi outi outi outi

    ; Обновить хэш
    ld (current_hash), a
    ret
```

   Результат: Если два вызова идут к одному драйверу подряд — переключение произойдет только при первом вызове.

1.11. Соглашения о регистрах: Естественность для Z80

1. Передача параметров

```
; HL, DE, BC — в порядке приоритета
ld hl, buffer    ; Основной параметр
ld bc, 256       ; Дополнительный
call video_fill

; Иногда только A
ld a, COLOR_RED
call video_set_color
```

2. Возврат результата и ошибок

Гениальность в простоте:

```
; Успех: A=0, Carry сброшен
xor a  ; A=0, Carry=0
ret

; Ошибка: Акод=, Carry установлен
ld a, ERR_TIMEOUT
scf    ; Set Carry Flag
ret
```

Почему это удобно:

```
call disk_read
ret c           ; Выход при ошибке
; Продолжаем если OK

; Или цепочка вызовов:
call init_video
call c, .error
call init_audio
call c, .error
call init_input
call c, .error
```

Стандартные коды ошибок:

```
ERR_SUCCESS      equ 0
ERR_NOT_SUPPORTED equ 1
ERR_TIMEOUT      equ 4
ERR_NO_DEVICE    equ 11   ; Устройство отсутствует
```

1.12. Пример из реальной жизни: Аудиодрайвер для YM2149

```
; ===========================================================================
; Драйвер звукового чипа YM2149
; ===========================================================================

SECTION DRV_VTABLE
ym2149_vtable:
    jp ym_detect      ; +0
```

```
    jp ym_install     ; +3
    jp ym_get_info    ; +6
    jp ym_command     ; +9
    jp ym_uninstall   ; +12
    jp ym_play_note   ; +15 специфичный( метод)

; ---------------------------------------------------------------------
; Зависимости: нет, самостоятельное устройство
; ---------------------------------------------------------------------
DEPENDS_ON:
    db 0

; ---------------------------------------------------------------------
; Данные
; ---------------------------------------------------------------------
SECTION DRV_DATA

ym_mem_map:
    db 0xC1        ; Хэш конфигурации
    db 2, 3, 4, 5  ; Банки памяти

ym_info:
    dw .name
    db 1, 2        ; Версия 1.2
    dw API_HAS_IRQ | API_STATIC_DEVICE
    db 0x38        ; Вектор прерывания
.name:
    db "YM2149 PSG v1.2", 0

; ---------------------------------------------------------------------
; Код драйвера
; ---------------------------------------------------------------------
SECTION DRV_CODE

ym_detect:
    ; Проверяем наличие YM2149
    call require_memory

    ; Пробуем записатьсчитать/ тестовое значение
    ld a, 0xFF
    out (YM_REG), a
    in a, (YM_REG)
    cp 0xFF
    jr nz, .not_found

    ; Устройство найдено
    ld a, 1
    ret

.not_found:
    xor a
    ret
```

```
ym_command:
    ; Обработка команд IOCTL
    cp CMD_RESET
    jr z, .reset
    cp CMD_AUD_PLAY
    jr z, .play_cmd
    cp CMD_AUD_STOP
    jr z, .stop_cmd

    ; Неизвестная команда
    ld a, ERR_NOT_SUPPORTED
    scf
    ret

.reset:
    ; Сброс звукового чипа
    xor a
    ld bc, 13
    ld hl, ym_registers
.reset_loop:
    out (c), a
    inc c
    djnz .reset_loop
    xor a
    ret

.play_cmd:
    ; HL = данные ноты
    ld a, (hl)
    out (YM_FREQ_A), a
    inc hl
    ld a, (hl)
    out (YM_FREQ_A_HI), a
    xor a
    ret

; Быстрый обработчик прерываний
; Сохраняет только то, что использует
ym_irq_handler:
    push af
    push hl

    ; Обновляем частоты
    ld hl, (ym_freq_ptr)
    ld a, (hl)
    out (YM_FREQ_A), a

    ; Следующий байт
    inc hl
    ld (ym_freq_ptr), hl

    pop hl
    pop af
```

```
    ei
    ret
```

1.13. Структура проекта: Как это всё собирается

```
project/
├── core/
│   ├── kernel.asm       * 1000 строк: планировщик, IRQ
│   ├── memory.asm       * 500 строк: require_memory, банки
│   └── loader.asm       * 300 строк: загрузка драйверов
├── drivers/
│   ├── video/crtc.asm   * 1500 строк: весь драйвер CRTC
│   ├── audio/ym2149.asm * 800 строк: весь драйвер YM2149
│   └── input/kbd.asm    * 600 строк: вся клавиатура
├── services/
│   ├── fs/fat16.asm     * Файловая система
│   └── console/text.asm * Текстовый вывод
└── build/
    └── memmap.cfg       * Кто где живёт
```

Конфигурация memmap.cfg:

```
# Фиксированные слоты
0xFD00: drivers/audio/
0xFD20: drivers/video/
0xFD40: drivers/input/
0xFD60: services/fs/
```

Заключение: Что мы получили

Мы создали систему, которая:

1. **Быстрая** — фиксированные адреса, lazy MMU, гибкие обработчики IRQ
2. **Гибкая** — IOCTL через `command`, диапазоны команд
3. **Надёжная** — явные зависимости, hot-detect для сменных устройств
4. **Простая** — один файл на драйвер, понятные соглашения

Это не академический стандарт. Это набор практик, выстраданных на реальном железе, где каждый такт на счету, а память измеряется килобайтами.

Код, написанный по этому стандарту, будет работать на Aleste через 30 лет — потому что он основан на простых, проверенных временем принципах, а не на модных абстракциях.